

# Omok - Adversarial Search algorithm

2020170831 민찬홍

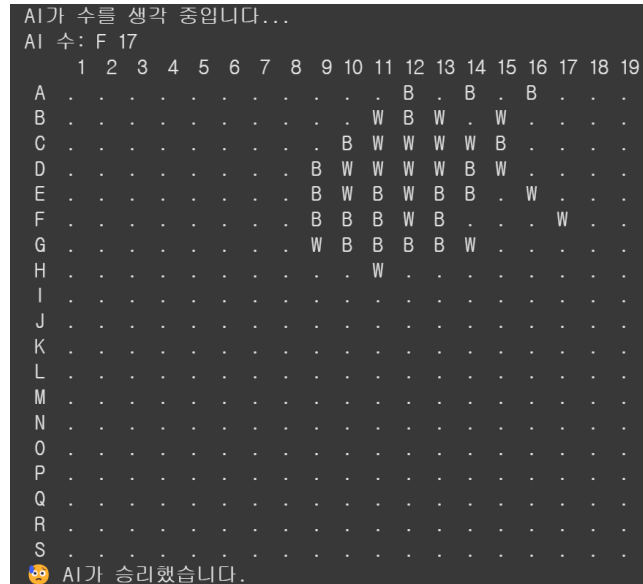


그림 1 B: 사용자 W: AI

## 1. Search Algorithm: Iterative Deepening Alpha-Beta Pruning

본 오목 AI는 **Iterative Deepening Alpha-Beta Search** 알고리즘을 기반으로 동작한다. 이 알고리즘은 다음과 같은 방식으로 수행된다:

- **Iterative Deepening:** 시간 제한(15초) 내에서 가능한 최대 깊이까지 반복적으로 탐색을 수행한다.
- **Alpha-Beta Pruning:** 탐색 도중 최적의 결과를 내지 못할 것으로 판단되는 경로는 가지치기를 통해 탐색에서 제외한다.  
이 때, 최적화를 위해 이전 탐색 단계에서 가장 유망했던 수를 우선적으로 탐색하여 pruning 효과를 극대화한다 (move ordering 적용).

이를 통해 AI는 제한된 시간 내에 효율적으로 최선의 수를 선택할 수 있다.

## 2. Heuristic Evaluation Function (휴리스틱 평가 함수)

AI는 현재 오목판에서 돌의 배치를 분석하고 점수를 부여하여 각 상태를 평가한다. 평가 방식은 다음과 같다:

## 공격 및 수비 점수 계산

공격 점수는 플레이어가 만든 유리한 패턴 수에 비례하여 증가한다. 수비 점수는 상대방이 만든 위험한 패턴 수에 비례하여 감소하며, 1.5배 가중치를 부여하여 더 강하게 페널티를 적용한다.

공격보다 수비에 더 큰 가중치를 부여하는 이유는 수비를 못하면 바로 패배로 직결되기에 실수를 더 치명적으로 본다는 의미로 상대 수에 더 큰 가중치를 두기 위함이다. 동시에 미래의 위험한 수를 미리 차단하기 위한 목적도 존재한다.

**주요 패턴 및 점수 예시**(.은 수가 놓이지 않은 상태(=EMPTY)를 의미함)

| 패턴              | 의미    | 점수             |
|-----------------|-------|----------------|
| BBBBB           | 5목 완성 | 10,000,000     |
| .BBBB.          | 열린 4  | 500,000        |
| BBBB. / .BBBB   | 닫힌 4  | 100,000        |
| .BBB.           | 열린 3  | 20,000         |
| .BB.B. / .B.BB. | 끊어진 3 | 8,000 ~ 15,000 |
| .BB.            | 열린 2  | 3,000          |

위 표와 같이 가능한 주요 패턴을 최대한 고려하여 중요도에 따른 점수를 차등 부과하는 heuristic evaluation function을 구현하였다.

## 기대 효과

수비에 더 높은 중요도를 부여하여 패배로 이어질 수 있는 상황을 사전에 차단 가능함으로써 지속적인 게임이 가능하다. 또한, 다양한 열린 형태와 끊어진 형태의 패턴들을 포함하여 보다 정교한 전략적 평가가 가능하도록 구성하였다.

### 3. Source Code 구조

#### OmokGame Class

- board: 19x19 크기의 오목판 상태를 저장하는 2차원 리스트
- place\_stone(x, y, color): 해당 좌표에 돌을 두며, 흑돌의 삼삼 금지 여부를 검사한다. 오목 공식 룰에 의하면 삼삼 금지는 먼저 두는 흑돌에 패널티를 부과하기 위한 규칙이라고 명시되어 있기에 삼삼 금지는 백돌에게는 적용되지 않는다.
- get\_actions(current\_color): 현재 가능한 수를 반환하며, 기본적으로 기존 돌 주변 2칸 범위 내의 빈 칸을 후보로 고려한다. 이는 19x19 오목판(총 361칸) 전체를 탐색하는 방식에 비해 연산량을 획기적으로 줄여준다. 전체 빈 칸을 모두 탐색하면 후보 수가 지나치게 많아져 탐색 트리의 가지 수가 폭발적으로 증가하고, 결과적으로 AI가 제한 시간 내에 좋지 못한 수를 결정하는 결과를 초래하기 때문에 이와 같이 탐색 범위를 제한하였다. 실제 사람도 대부분 돌 주변에 수를 두는 전략을 따르므로, 이 방식은 실전성과 효율성을 동시에 확보할 수 있다.

추가로, get\_actions 함수는 단순히 주변 칸만 보는 것이 아니라, 특정 위협 수 예측도 함께 수행한다. 기존 돌 주변 2칸 범위 내 위협 수가 존재하지 않는 경우 탐색을 하지 않아 패배로 직결되는 현상을 방지하기 위함이다. 예를 들어 상대방이 연속으로 3개의 돌을 놓은 .BBB.와 같은 패턴이 존재하면, 해당 패턴의 양 끝이 비어 있는 경우 이를 강제로 후보 수에 포함시킨다. 위협 수는 코드상에서 threat\_patterns = [".BBB.", ".B.BB.", ".BB.B.", ".B.B.B.", "BBBB.", ".BBBB."]로 지정하였다.

이와 같은 전략은 공격 수를 사전에 차단하거나 방어 수를 선제적으로 고려하게 하여 AI의 판단력을 향상시킨다. 후보 수가 제한되면 알파-베타 탐색의 가지치기 효과가 극대화되며, move ordering 방식과 결합되어 pruning 효율이 더욱 향상된다. 그 결과 제한된 시간 내에 더 깊이 있는 수읽기를 가능하게 하며, 실전에서 최선에 가까운 플레이를 구현할 수 있게 되는 것이다.

- evaluate(color): 전략적 패턴 기반으로 오목판 상태의 점수를 계산한다. 현재 플레이어와 상대 플레이어 각각의 돌 배치에 대해 점수를 계산하며, 공격적인 패턴은 가산점으로, 수비적 위험은 감점 요소로 처리한다. 이 함수는 다양한 전술 패턴(예: 5목, 열린 4, 열린 3, 끊어진 3 등)에 가중치를 부여하여 현재 상태의 유리함 또는 위기 상황을 정량화한다. 상대의 위협 수에 대해서는 1.5배의 패널티를 적용하여, 방어 수를 더 우선시하도록 설계하였다. 이를 통해 AI는 승리를 위한 수뿐 아니라 패배를 막기 위한 수 또한 적극적으로 고려하게 된다. 이는 지속적인 게임이 가능하기 위한 필수 조건이다.

- `violates_sam_sam(x, y, color)`: 삼삼 금지 규칙을 검사한다. 본 규칙은 흑돌에게만 적용되며, 두 개의 열린 삼(예: .BBB.) 형태가 동시에 만들어지는 수를 금지한다. 이 함수는 네 가지 방향(가로, 세로, 대각선 2개)에 대해 탐색을 수행하며, 각 방향에서 열린 삼이 만들어지는지 문자열 패턴으로 검사한다. .BBB., .BB.B., .B.BB., .B.B.B. 등 전형적인 삼삼 형태를 판별하여, 두 개 이상의 열린 삼이 동시에 생기는 경우 해당 수는 불가하다고 판단하고 착수를 허용하지 않는다. 이를 통해 흑돌의 과도한 선공 이점을 제어하고, 규칙에 따라 공정한 게임 흐름을 보장한다.
- `check_winner(color)`: 해당 색깔의 돌이 5개 연속으로 놓였는지 검사한다.

### 탐색 알고리즘

- `alpha_beta_search(game, color, time_limit)`: 주어진 시간 제한 내에서 iterative deepening 방식으로 탐색을 수행한다.
- `iterative_deepening(game, color, depth, ...)`: max/min 노드를 재귀적으로 탐색한다.
  - `max_value()` / `min_value()` 함수를 통해 알파-베타 가지치기를 적용한다.
  - move ordering을 통해 pruning 효율을 높인다. 구체적으로는, 이전 탐색 깊이에서 가장 높은 평가를 받은 수를 다음 탐색 깊이에서 우선적으로 탐색하도록 수의 순서를 재정렬한다. Alpha-Beta 탐색에서는 가능한 수 중 더 나은 수를 먼저 평가할수록 가지치기(pruning)가 조기에 일어나 탐색 범위를 줄일 수 있다. 예를 들어 첫 번째 수에서 좋은 결과가 나오면 이후의 수들은 탐색 전에 이미 잘릴 가능성이 높아진다. 따라서 move ordering을 적용하면 의미 없는 수에 대한 탐색을 최소화할 수 있고, 그만큼 더 깊이 있는 수읽기가 가능해진다. 이는 특히 제한 시간이 있는 환경에서 탐색 효율성과 전략적 정확성을 동시에 향상시키는 데 매우 중요한 역할을 한다.

### UI 및 게임 루프

- `display_board()`: 알파벳(A~S)과 숫자(1~19)로 구성된 텍스트 기반 오목판을 출력한다. 알파벳과 숫자는 각각 행과 열을 위치를 의미하게 된다.
- `play_game()`: 사용자와 AI가 번갈아 가며 수를 두는 게임 루프를 실행한다.
  - 입력 형식: A 3
  - 사용자와 AI 모두 수를 두는 데 최대 15초의 제한 시간이 있다. 제한 시간을 초과하게 되면 상대방 턴으로 넘어가게 된다.

## 4. 실행 방법

### 필요 환경

- Python 3.x 이상 (Google Colab 또는 로컬 실행 환경)

### 실행 절차

1. 본 스크립트를 실행한다.
2. 게임 시작 시 흑(B) 또는 백(W)을 선택한다.
3. 사용자와 AI가 번갈아가며 수를 둔다.
4. AI는 각 수를 결정하는 데 최대 15초를 사용한다.

### 입력 예시

당신의 수를 입력하세요 (예: A 3):

- A~S (행), 1~19 (열)의 좌표 형식으로 입력한다.

## 5. 요약 정리

- **탐색 알고리즘:** Iterative Deepening + Alpha-Beta Pruning + Move Ordering
- **평가 함수:** 전략 패턴 기반의 공격/수비 휴리스틱 적용
- **기능 특징:** 삼삼 금지, 시간 제한(15초), 사용자 돌 선택 기능, 위협 수 감지 포함
- **게임 구성:** 사용자 대 AI 텍스트 기반 오목 대국 프로그램

위와 같은 방식으로 구현된 본 AI는 전략적인 수읽기와 효율적인 탐색을 통해 강력한 오목 플레이를 수행한다. 실제로 첫 번째 페이지 그림에서 알 수 있듯이 약 8분의 혈투 끝에 AI에게 패배한 것을 확인할 수 있다.